Practice Set: 07

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define **Comments** in C++ and state their primary purpose. Explain how the compiler preprocessor pass handles single-line (//) and multi-line (/* */) comments in terms of RAM allocation, machine code generation, and runtime performance.

Question 2: Explain the four fundamental categories of **Control Instructions** in C++: Decision Control, Iterative Control, Switch-Case Control, and Unconditional Jump Control.

Question 3: What is the **Golden Rule of Truth Evaluation** in C++ when evaluating non-boolean numeric expressions or pointer variables inside conditional contexts?

Question 4: Explain the common **Assignment vs. Comparison Bug** (e.g., if (x = 0) vs. if (x == 0)) in C++ selection statements and explain how each evaluates in memory.

Question 5: Compare **while**, **do-while**, and **for** loops across three criteria: Control Category (Entry-Controlled vs. Exit-Controlled), Condition Check Location, and Minimum Guaranteed Iteration Count.

Question 6: Explain the execution mechanics and lifecycle steps required for a well-designed loop (Initialization, Condition Test, Update/Step). What causes an **Infinite Loop** in a while construct?

Question 7: Detail the functional behavior differences between the loop interrupt statements **break** and **continue** using a flow diagram or step explanation.

Question 8: What are the five strict rules governing **switch-case** blocks in C++ regarding expression data types, case label constants, label uniqueness, fall-through mechanics, and the default label?

Question 9: Why does nesting multi-line block comments (/* /* */ */) cause a compilation error in C++? What is the purpose of the modern C++17 [[fallthrough]] attribute?

Question 10: Write a complete C++ program that demonstrates a for loop skipping even numbers using continue, aborting at a specific threshold using break, and implementing a basic switch statement with integer/character case branches.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): How much RAM memory do source code comments consume in the generated machine binary executable?

Question 12 (Viva): What is the minimum number of times a do-while loop executes if its test condition is false from the start?

Question 13 (Viva): Why does placing an accidental semicolon immediately after a loop header (e.g., while (i > 0);) cause a program execution freeze?

Question 14 (Viva): Can floating-point data types like float or double be used as controlling expressions in a switch statement in C++?

Question 15 (Viva): What is "Unintended Fall-Through" in a switch-case construct, and how is it prevented?

Question 16 (Viva): What compiler optimization structure is commonly constructed for a switch block instead of evaluating sequential conditions like an if-else-if ladder?

Question 17 (Viva): Why is the indiscriminate use of the goto statement strongly discouraged in modern C++ software engineering?

Question 18 (Viva): What is the truth value of evaluating a negative integer like -5 or -100 inside an if condition in C++?

Question 19 (Viva): Why is enclosing single-line statements in curly braces {} considered a best practice even when {} is optional?

Question 20 (Viva): In a for loop header for (init; condition; update), how many times is the initialization expression executed during the entire loop lifecycle?

Answers & Explanations

Answer: 01

Definition & Purpose: Comments are non-executable text annotations written to explain code intent, document complex logic, state design assumptions, or store metadata.

Preprocessor Handling: During lexical analysis, the preprocessor strips all single-line (//) and multi-line (/* */) comments, replacing them with plain whitespace.

Overhead: Comments consume **0 bytes of RAM** in the output executable, generate zero machine instructions, and have zero impact on runtime performance.

Answer: 02

Decision Control Instructions (Selection): Branch execution paths based on true/false conditional checks (if, if-else, if-else-if ladder).

Iterative Control Instructions (Loops): Repeat execution of code blocks while conditions remain valid (while, do-while, for).

Switch-Case Control Instructions: Provide multi-way jump branching based on integral/enumerated value matches.

Unconditional Jump Control Instructions: Redirect execution unconditionally (goto, break, continue, return).

Answer: 03

The **Golden Rule of Truth** in C++ evaluates conditional states as follows:

TRUE: Any non-zero numeric value (positive or negative, e.g., 1, -5, 100) or non-null pointers.

FALSE: Value of exactly zero (0, 0.0, nullptr, false).

Answer: 04

if (x = 0) (Assignment): Stores 0 into variable x. Because the expression evaluates to 0, it is treated as false, causing the block to never execute regardless of x's initial value.

if (x = 5) (Assignment): Stores 5 into x. Because 5 is non-zero, it always evaluates to true.

if (x == 0) (Comparison): Compares the value in x against 0 using the equality operator without mutating x.

Answer: 05

Loop Type	Control Category	Condition Check Location	Minimum Iterations
while	Entry-Controlled	Top (Before executing loop body)	0 Times (Skipped if initially false)
do-	Exit-	Bottom (After	1 Time (Guaranteed to

Loop Type	Control Category	Condition Check Location	Minimum Iterations
while	Controlled	executing loop body)	run at least once)
for	Counter-Driven	Top (Consolidates init, test, update)	0 Times

Answer: 06

Loop Lifecycle Steps:

Initialization: Sets initial values for counter/tracking variables.

Condition Test: Evaluates boolean truth to determine whether iteration proceeds.

Update/Step: Modifies tracking variables toward a terminating threshold.

Infinite Loop Cause: Omitting the update/step modification inside the loop body leaves the condition permanently true, causing the program to hang in an infinite loop.

Answer: 07

break: Instantly terminates the enclosing loop or switch block and transfers execution directly to the statement immediately following the structure.

continue: Skips the remaining statements within the current iteration and jumps directly to the loop condition test/update step to begin the next cycle.

Answer: 08

Integral/Enum Types Only: The controlling switch expression must evaluate to an integral type (int, char, short, long) or an enum class (floating-point types like float/double are illegal).

Constant Case Labels: Case labels must be compile-time constants or literals (e.g., case 5:); variable expressions cause compilation errors.

Unique Labels: Every case label value in a single switch block must be strictly unique.

Fall-Through Mechanics: Execution cascades sequentially through subsequent case statements unless halted with a break statement.

default Label: An optional fallback label executed when no case constants match the evaluated expression.

Answer: 09

Block Nesting Error: Block comments cannot be nested because the very first */ encountered terminates the entire comment block, causing the remainder of the comment to be parsed as invalid syntax.

[[fallthrough]] (C++17): A standard attribute annotation used to explicitly indicate that a fall-through to the next case in a switch statement is intentional, suppressing compiler diagnostics and warnings.

Answer: 10

```
#include <iostream>
int main()
 {
   // 1. Loop with continue and break
   std::cout << "Odd numbers up to threshold:\n";
   for (int n = 1; n <= 10; n++)
   {
      if (n % 2 == 0)
       {
         continue; // Skip even numbers
      }
      if (n == 7)
     {
         break;    // Abort loop when reaching 7
      }
      std::cout << n << " ";
   }
   std::cout << "\n\n";

   // 2. Switch block demonstration
   char userGrade = 'B';
   switch (userGrade)
   {
      case 'A':
         std::cout << "Excellent (Score >= 90%)\n";
         break;
      case 'B':
         std::cout << "Good (Score >= 80%)\n";
         break;
      default:
         std::cout << "Other Grade\n";
         break;
```

```
    }

    return 0;
}
```

Answer: 11
Comments consume **0 bytes** of RAM in the compiled binary because they are stripped out completely by the preprocessor before compilation occurs.

Answer: 12
It executes a minimum of **1 time**, because condition evaluation occurs at the exit (bottom) after the loop body runs.

Answer: 13
An accidental semicolon creates a null (empty) loop body; the loop continuously checks an unmodified condition without executing updates, causing an immediate infinite freeze.

Answer: 14
No; C++ switch statements strictly require integral types (int, char, short, long) or enumeration types.

Answer: 15
It occurs when a break statement is omitted at the end of a matched case, causing execution to cascade into subsequent case blocks. It is prevented by terminating each case with break.

Answer: 16
A **Jump Table** (a direct memory branch offset based on index values), which executes branch selections efficiently.

Answer: 17
goto creates unstructured "spaghetti code" that obscures control flow, bypasses object destructors, and causes severe debugging problems.

Answer: 18
It evaluates to **true**, because C++ treats every non-zero numeric value (positive or negative) as true in boolean contexts.

Answer: 19
It prevents dangling-else logic bugs and unintentional statement execution errors during future code refactoring.

Answer: 20
The initialization statement runs **exactly once** at the initial entry of the loop lifecycle.